

Reviewer Pack — Coxeter Matrix Invariant vs BP_ReadTwice Circuit Size

Entry 7a9df8c9860d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-23 13:53:57 UTC

Coxeter Matrix Invariant vs BP_ReadTwice Circuit Size

Entry ID: 7a9df8c9860d **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-23 13:53:57 UTC

1. Conjecture

Field A (mathematical branch): Coxeter Theory **Field B** (complexity object): Branching Program Complexity

Statement:

{'para1': 'For a given read-twice branching program P computing the IP_2 function, the Coxeter matrix invariant of the associated Coxeter group G is upper bounded by a polynomial in the size of P.', 'para2': 'Specifically, there exists a constant c such that $\rho(G) \leq c * \log(\text{size}(P))$ for all read-twice BP P.', 'para3': 'For the trivial IP_2 function computed by a read-twice BP, the Coxeter matrix invariant of G is at least $\Omega(n)$.'}

Rationale (proposer's reasoning):

{'para1': 'Coxeter groups are known to encode geometric structures and symmetry properties, which may provide insights into the complexity of certain computations.', 'para2': 'The connection between Coxeter theory and computational complexity could expose new structural properties of branching programs.', 'para3': 'An invariant that captures the inherent difficulty of a problem might exist within the algebraic structure of Coxeter groups.'}

Taxonomy category: BP_READTWICE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): c834ca438d10b4cb

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For the conjecture on Coxeter Matrix Invariant vs BP_ReadTwice Circuit Size, support will be indicated by a Pearson correlation coefficient of ≥ 0.7 between $\log(\text{size}(P))$ and $\rho(G)$ across at least 30 seeds, with no seed showing $\rho(G) > 10 * \log(\text{size}(P))$. Falsification will occur if any seed demonstrates $\rho(G) > 10 * \log(\text{size}(P))$ or the Pearson correlation coefficient < 0.5 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	0.90	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.70	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.0s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_bp(n):
        # Generate a random read-twice branching program for IP_2 with size n
        bp = []
        for _ in range(n):
            if random.choice([True, False]):
                bp.append(random.randint(0, 1))
            else:
                bp.append((random.randint(0, 1), random.randint(0, 1)))
        return bp

    def ip2_function(bp):
        # Evaluate the IP_2 function for a given read-twice branching program
        result = 0
        for bit in bp:
            if isinstance(bit, tuple):
                result += bit[0] * bit[1]
            else:
                result += bit
        return result

    def coxeter_group(bp):
        # Generate the Coxeter group associated with a read-twice branching program
        n = len(bp)
        G = [[0] * n for _ in range(n)]
        for i in range(n):
            if isinstance(bp[i], tuple):
                G[i][bp[i][0]] = 1
                G[i][bp[i][1]] = 1
```

```

        G[i][i] = 1
        G[i][(i + 1) % n] = 2
        G[(i + 1) % n][i] = 2
    else:
        G[i][i] = 1
    return G

def coxeter_matrix_invariant(G):
    # Compute the Coxeter matrix invariant of a Coxeter group
    n = len(G)
    det = determinant(G, n)
    if det == 0:
        return float('inf')
    return abs(det) ** (1 / n)

def determinant(matrix, n):
    if n == 1:
        return matrix[0][0]
    det = 0
    for j in range(n):
        submatrix = [row[:j] + row[j+1:] for row in matrix[1:]]
        det += ((-1) ** j) * matrix[0][j] * determinant(submatrix, n - 1)
    return det

def log_size(bp):
    # Compute the logarithm of the size of a read-twice branching program
    return math.log(len(bp))

def is_trivial_bp(bp):
    # Check if a read-twice branching program computes the trivial IP_2 function
    return all(isinstance(bit, int) for bit in bp)

n = random.randint(5, 40)
bp = generate_bp(n)
G = coxeter_group(bp)
rho_G = coxeter_matrix_invariant(G)
log_p = log_size(bp)

if is_trivial_bp(bp):
    conjecture_holds = rho_G >= n
else:
    conjecture_holds = rho_G <= 10 * log_p

return {
    "metric_name": "Coxeter Matrix Invariant vs BP_ReadTwice Circuit Size",
    "metric_value": rho_G,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": "" if conjecture_holds else "Trivial IP_2 function"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(2, 97) for _ in range(30)]

    results = []

```

```

for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result)

mean_value = sum(r["metric_value"] for r in results) / len(results)
std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif any(r["metric_value"] > 10 * math.log(len(bp)) for bp, r in zip(results)):
    first_failing_seed = next(i for i, r in enumerate(results) if r["metric_value"] > 10 * math.log(len(bp)))
    print(f"RESULT: FALSIFIED counterexample=\"Trivial IP_2 function\" first_failing_seed={first_failing_seed}")
else:
    print(f"RESULT: INCONCLUSIVE reason=insufficient_evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it did not complete its execution to provide a Pearson correlation coefficient. | next: Re-run the test with increased time limits or optimize the code to ensure it completes within the given time frame.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14761
2	preregistration	ollama_remote	glm4:latest	0	0	10157
3	novelty	ollama_remote	glm4:latest	0	0	8663
4	novelty	ollama_remote	glm4:latest	0	0	8536
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16505
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12693
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16129
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11799
9	judge	ollama_remote	glm4:latest	0	0	19732

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 118974 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/7a9df8c9860d.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/7a9df8c9860d.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/7a9df8c9860d.tar.gz (if generated)